SafeExec: A Hardened, Threat-Modeled Python Execution Sandbox for LLM-Agent Tool-Use

AI-use disclosure. *Drafted with Codex (GPT-5, Codex desktop app) from student-authored project artifacts, test outputs, prior Canvas submissions, and repository evidence generated in this project. AI-drafted, student-revised. Full audit trail:* `docs/ai-use-log.md`.

Student: Zixuan Liang

Program: M.S. Computer Information Sciences, Harrisburg University

Course: CISC 699-50-A-2026/Summer - Applied Project in Computer Information Sciences

Advisor: Prof. Khalid Lateef

Instructor: Dr. Majid Shaalan

Draft date: 2026-06-26

Draft status: Near-final technical report draft for Hard Stop 6; final corpus expansion and final benchmark refresh remain.

Abstract

Large language model agents increasingly execute generated code as part of tool-use workflows. This creates a systems-security boundary where untrusted model output becomes operating-system process execution. SafeExec is a compact, reproducible Python execution sandbox designed to evaluate that boundary rather than merely demonstrate it. The artifact provides a synchronous execution API, a development local-subprocess backend, a hardened Docker backend, and a gVisor runtime path through Docker's `runsc` runtime. The evaluation approach ties the implementation to a threat model, functional tests, containment-oriented probes, benchmark timing data, environment snapshots, reproducibility checks, and an explicit issue log.

At this draft stage, SafeExec has a runnable source package, documented setup path, API shell, command-line smoke path, unit tests, local validation workflow, Docker/gVisor target-host benchmark evidence, and a clean-run reproducibility package. Fresh W12 local evidence passed smoke, unit, validation, and

reproducibility-audit checks. Earlier target-host evidence produced 130 records: local 30/30, hardened Docker 50/50, and gVisor 49/50. The key technical finding so far is positive but bounded: the measurement path is working, Docker is stable on the current probe set, and gVisor is integrated, but final claims must still wait for larger HumanEval/MBPP functional subsets, an expanded adversarial manifest, and a final benchmark protocol that separates cold-start overhead from program execution time.

1. Problem Statement and Significance

LLM agents are useful because they can plan, call tools, interpret output, and continue working. They are risky for the same reason. When an agent generates or modifies code and sends it to an execution tool, natural-language uncertainty crosses into process execution. Prompt injection, excessive agency, or an ordinary model error can turn into file access, network egress, resource exhaustion, or host-environment exposure if the executor is weak.

SafeExec addresses one narrow version of that problem: Python code execution for LLM-agent tool use on a controlled Linux host. The project does not claim to solve prompt injection, production multitenancy, authentication, or cloud operations. Instead, it asks whether a small academic artifact can make the code-execution boundary observable: What backend ran the code? Which limits were applied? Was output structured? Were network, filesystem, and resource controls exercised? Could another reader reproduce the evidence?

This is significant because many sandbox discussions are product-oriented or architecture-oriented but do not expose the evaluation path. Managed systems such as OpenAI Code Interpreter, Anthropic code execution, E2B, and Modal Sandboxes show that sandboxed code execution is now a mainstream agent capability. SafeExec's contribution is smaller and more inspectable: a threat-modeled execution service plus reproducible evidence comparing hardened Docker and gVisor on the same request and result model.

2. Related Work and Background

The project is grounded in four source groups. NIST's adversarial machine learning taxonomy and OWASP's LLM Top 10 frame LLM tool-use risks in terms of attacker goals, excessive agency, prompt injection, and unsafe downstream actions [1], [2]. Greshake et al. show how indirect prompt injection can compromise LLM-integrated applications through remote content [3]. These sources justify treating generated code as untrusted even when the visible user intent seems benign.

Linux isolation and container-hardening sources shape the implementation. Seccomp-BPF, namespaces, cgroups, capabilities, non-root users, read-only root filesystems, and network namespace controls are complementary, not interchangeable [5]. Historical runc vulnerabilities such as CVE-2019-5736 and CVE-2024-21626 support the project's refusal to treat default Docker as a complete adversarial boundary [7], [8]. In SafeExec, Docker is useful only when its hardening choices are visible and tested.

gVisor and Firecracker clarify the design tradeoff. gVisor moves much of the system-call surface into a userspace Sentry, reducing direct host-kernel exposure at the cost of compatibility and overhead [4], [6]. Firecracker is outside the implementation scope, but its microVM model explains why stronger isolation may be attractive for future work [9].

Finally, evaluation literature shapes the evidence plan. SWE-bench emphasizes execution-based evaluation rather than subjective claims [14]. HumanEval and MBPP provide functional Python task sources that can seed benign correctness testing [17], [18]. Reproducibility guidance from Pineau et al. supports capturing command lines, versions, sample counts, host metadata, and limitations [15], [16].

3. Requirements and Scope

SafeExec's approved scope is intentionally narrow:

- Python 3.11 code execution only.
- Single-tenant research artifact, not production service.

- No outbound network from sandboxed code.
- Ephemeral per-request workspace.
- Structured execution results with stdout, stderr, exit code, duration,

backend, containment flag, containment reason, and applied limits.

- Hardened Docker as the primary container baseline.
- gVisor through `runsc` as the stronger-isolation comparison path.
- Functional tests, containment-oriented probes, benchmark records, and

reproducibility evidence submitted as first-class artifacts.

The main success criteria remain:

- Runnable service/API path with reproducible setup.
- Functional-correctness evidence for benign Python programs.
- Containment evidence for resource, filesystem, network, and runtime boundary

probes.

- Comparative Docker/gVisor evidence with timing and limitations.
- Transparent AI-use, issue, and reproducibility logs.

4. System Design

SafeExec is built around one request model and one result model. A client sends

Python code and backend/limit choices. The service validates the request,

selects a backend, executes the program, and returns a structured result.

```
Client or test harness
  -> ExecutionRequest(code, backend, limits)
    -> safeexec.service.execute_code()
      -> LocalSubprocessBackend | DockerBackend | DockerBackend(runtime="runsc")
        -> ExecutionResult(status, stdout, stderr, exit_code, duration, limits)
```

The local backend is explicitly a development shim. It uses a temporary

directory and an isolated Python interpreter invocation (`python -I`), but it is

not a security boundary. Its purpose is to let the API, result schema, smoke

tests, and validation harness run on the macOS authoring machine.

The Docker backend constructs a hardened command with:

- `--network none`
- CPU, memory, and PID limits
- read-only root filesystem
- dropped Linux capabilities
- `no-new-privileges`
- tmpfs-mounted `/tmp`
- non-root user `65534:65534`

- `python:3.11-slim` as the runtime image

The gVisor backend reuses the same Docker command shape but adds `--runtime runsc`. This keeps the comparison methodologically clean: the same request, code, limits, image, and result schema are used, with only the runtime boundary changed.

5. Implementation Status

The current repository contains:

- `src/safeexec/models.py`: request, limit, and result dataclasses.
- `src/safeexec/service.py`: backend selection and execution entry point.
- `src/safeexec/backends/local_subprocess.py`: development local backend.
- `src/safeexec/backends/docker.py`: hardened Docker/gVisor command builder and executor.
- `src/safeexec/api/server.py`: simple JSON `/health` and `/execute` HTTP API.
- `scripts/smoke_safeexec.py`: smoke execution path.
- `scripts/run_validation_workflow.py`: repeatable local/API validation.
- `scripts/run_midpoint_evidence.py`: Docker/gVisor benchmark and probe harness.
- `scripts/audit_reproducibility.py`: reproducibility audit.
- `scripts/package_artifact.py`: source package builder.
- `scripts/run_final_evidence.py`: W12 final evidence bundle runner.

The code is deliberately compact. This is a strength for the academic artifact: there are fewer hidden pathways, and each result can be traced back to a small number of source files. The main limitation is that the final functional corpus and adversarial manifest are not yet large enough for final quantitative claims.

6. Evaluation Methodology

SafeExec's evidence is layered:

1. **Smoke test:** run a minimal program through the service layer and inspect structured output.
2. **Unit/functional tests:** verify local subprocess behavior, Docker command construction, API behavior, and validation workflow.

3. **Local/API validation workflow:** repeat deterministic local cases and a live local `/execute` API trace.
4. **Midpoint target-host benchmark:** run correctness, failure-control, and containment probes across local, Docker, and gVisor backends on the Linux target host.
5. **Reproducibility audit:** verify required docs, README setup markers, Make targets, dependency policy, and tool status.
6. **Clean package run:** build a source tarball, unpack it under `/tmp`, and run smoke, tests, validation, and audit outside the live working tree.

This methodology is designed to avoid the common failure mode of showing a screenshot of a successful output without controlled conditions. Each result table links to a command, output file, environment snapshot, or issue note.

7. Current Results

7.1 Fresh W12 local evidence

The W12 evidence bundle was generated by:

```
make final-evidence
```

The command wrote outputs under `docs/12-hard-stop-6/evidence/`.

Check	Result	Evidence
Smoke execution	PASS	`smoke-output.txt`
Unit/functional tests	PASS	`unit-tests-output.txt`
Local validation workflow	PASS	`validation-summary.txt`
Reproducibility audit	PASS	`reproducibility-audit.md`

The local validation result was:

```
include_docker: False
all_passed: True
local: 12/12 passed
api_trace: passed
```

7.2 W8 target-host Docker/gVisor evidence

The strongest current container evidence remains the W8 target-host benchmark:

Backend	Passed / Total	Pass rate	Mean duration	Median duration	Maximum duration

Local subprocess	30 / 30	100.0%	93.2 ms	32.9 ms	402.0 ms
Hardened Docker	50 / 50	100.0%	868.8 ms	771.7 ms	1985.4 ms
gVisor (`runsc`)	49 / 50	98.0%	1361.8 ms	1327.5 ms	3167.2 ms

Docker passed all current correctness, failure-control, and containment probes. gVisor passed all but one `tmpfs_write_allowed` run, which timed out at the current wall-time boundary. The interpretation is not "gVisor failed as a sandbox"; it is that timing methodology must distinguish container startup and program execution more carefully before final performance claims.

7.3 W10 reproducibility evidence

W10 added `.env.example`, `docs/reproducibility/`, `make repro-audit`, and `make package-artifact`. A clean-run package was built, unpacked under `/tmp`, and executed successfully. This matters because the instructor feedback on W5 specifically penalized referenced-but-unsubmitted evidence. The W10/W12 package now includes direct evidence files, not just prose references.

8. Discussion

The project is on track technically, but its final value depends on evidence quality rather than additional feature growth. The strongest signal so far is that the execution path is stable enough to collect comparable results. The same request model can drive local development checks, Docker runs, gVisor runs, API traces, and result summaries. That validates the design decision to keep the API and evaluation harness close together.

The Docker results are encouraging. The current command shape exercises no-network, non-root, read-only-root, tmpfs, timeout, output-limit, and structured-result behavior. However, the probe set is still small. Final Docker claims should use a larger adversarial manifest and should report category coverage rather than only aggregate pass rate.

The gVisor results are also encouraging but require careful wording. gVisor is integrated and mostly passing, but the measured overhead is higher and tail latency caused one timeout. The final report should avoid treating overhead as

an afterthought. For short LLM-agent code snippets, startup behavior is part of the user-facing cost, but it should be labeled separately from actual program runtime.

The biggest remaining weakness is corpus depth. HumanEval/MBPP subsets have been approved and planned, but the committed W12 evidence does not yet include the final subset manifest or >=100 functional programs. The adversarial taxonomy exists, but the full >=40-program suite is not finished. The final week should therefore focus on corpus completion, manifest quality, and final evidence refresh rather than adding new runtime features.

9. Ethics, Security, Privacy, and Broader Impact

SafeExec is a dual-use security artifact. Its purpose is defensive: reduce the risk of LLM-agent code execution by making isolation behavior measurable. The adversarial suite should avoid copying live exploit code and should frame each program as a contained probe with expected outcomes. Public disclosure should emphasize category-level behavior, limitations, and containment context.

The project uses no personal, FERPA-protected, HIPAA-regulated, proprietary, or confidential data. The current validation programs are student-authored. Future HumanEval/MBPP reuse must preserve upstream license and provenance notes.

AI assistance was used for drafting, code review support, documentation structure, and some implementation help. The student remains responsible for correctness, final wording, and all submitted artifacts. The final report must include the AI-use appendix and should not present AI-assisted text or code as wholly unaided work.

10. Limitations and Future Work

Current limitations:

- The local subprocess backend is not a security boundary.
- Docker/gVisor checks require the Linux target host; local macOS can run only local/API validation.
- The final HumanEval/MBPP subset manifest is not yet complete.
- The adversarial suite is not yet at the approved >=40-program target.

- gVisor timing evidence needs cold-start/warm-run separation.
- The HTTP API is intentionally minimal and lacks production auth/TLS/rate limiting.

Future work after the course could include Firecracker microVM support, a larger adversarial corpus, deeper syscall/category analysis, CI-based reproducibility checks, richer policy configuration, and a small agent-demo adapter that records tool-call transcripts without requiring a live LLM API for core evaluation.

11. Final-Week Work Plan

Before final submission, the remaining work is:

1. Complete the HumanEval/MBPP subset manifest with task IDs, provenance, and license notes.
2. Expand the adversarial manifest to the approved category and count targets.
3. Re-run Docker/gVisor evidence on the Ubuntu target host with final cases.
4. Separate cold-start and warm-run timing in the final benchmark table.
5. Update report figures, tables, and appendix references from final output.
6. Record the final demo walkthrough using the W12 deck and demo script.
7. Rebuild the final source package and confirm `make final-evidence` still passes.

References

[1] A. Vassilev, A. Oprea, A. Fordyce, H. Anderson, X. Davies, and M. Hamin, *Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations*, NIST Trustworthy and Responsible AI Report AI 100-2 E2025, 2025.

[2] OWASP Foundation, *OWASP Top 10 for Large Language Model Applications*, OWASP Gen AI Security Project, v2025 materials.

[3] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection," arXiv:2302.12173, 2023.

[4] The gVisor Authors, "Security Model," *gVisor Documentation*.

[5] The Linux Kernel documentation, "Seccomp BPF (SECure COMPuting with

filters)."
[6] E. G. Young, P. Zhu, T. Caraza-Harter, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, "The True Cost of Containing: A gVisor Case Study," USENIX HotCloud, 2019.
[7] Red Hat Product Security, "runc - Malicious container escape - CVE-2019-5736," 2019.
[8] GitHub Advisory Database, "runc vulnerable to container breakout through process.cwd trickery and leaked fds - CVE-2024-21626," GHSA-xr7r-f8xq-vfvv, 2024.
[9] A. Agache et al., "Firecracker: Lightweight virtualization for serverless applications," USENIX NSDI, 2020.
[10] OpenAI, "Code Interpreter," OpenAI API Documentation.
[11] Anthropic, "Code execution tool," Claude API Documentation.
[12] E2B, "E2B Documentation."
[13] Modal, "Sandboxes," Modal Documentation.
[14] C. E. Jimenez et al., "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?," ICLR, 2024.
[15] J. Pineau et al., "Improving Reproducibility in Machine Learning Research (A Report from the NeurIPS 2019 Reproducibility Program)," arXiv:2003.12206, 2020.
[16] J. Pineau, "The Machine Learning Reproducibility Checklist," v2.0, 2020.
[17] OpenAI, "HumanEval: Hand-Written Evaluation Set," GitHub repository.
[18] Google Research, "Mostly Basic Python Problems Dataset," GitHub repository.